

Project Assignment

GeoControl

Version 1.0.0 - Mar 2025

© Stefano Mancini, Luca Ardito, Maurizio Morisio, Tommaso Fulcini 2025

GeoControl

- GeoControl is a web application composed of:
 - A web server, which exposes HTTP/Rest/JSON APIs
 - A web client which interacts with the server through HTTP/Rest/JSON calls
 - A database, which saves the application data

GeoControl – Front End

- Not available for the Task 1
- Will be provided, fully implemented, for the second task (code implementation)

GeoControl – Back End

- Implemented in Node.js using Express and TypeORM
- Available in the project repository
- Instructions and architecture details are described in README.md
- Provides the implementation of the HTTP/REST/JSON APIs defined in the OpenAPI (Swagger) specification
- Task 1: only the Swagger UI is available
- Task 2: complete architecture + some example features as reference for your work

GeoControl – Database

- The default DBMS used in the mandatory part of the project is SQLite, a lightweight, file-based database.
- In Task 4 (optional), a change request will require:
 - Dockerizing all components: front-end, back-end, and database
 - Replacing SQLite with MySQL, a server-based relational DBMS, to support multi-component deployment

npm package manager

- GeoControl back end and front end use the npm package manager
- npm is a package manager for the JavaScript programming language. It is the default package manager for the JavaScript runtime environment Node.js
- It consists of a command line client, also called npm, and an online database of public and paid-for private packages, called the npm registry
- If you do not have it yet, please install npm on your local machine!

npm package manager

- When using the npm package manager, two important files are created:
- package.json
 - Contains project metadata, lists dependencies, and defines scripts to automate tasks
 - Some scripts are already provided in Assignment 1
 - In Assignment 2, additional scripts will be included to support development, debugging, and testing
 - See README.md for usage details
 - More info: [npm docs](https://docs.npmjs.com/)
- package-lock.json
 - Records the exact versions of all installed packages to ensure consistent installations across environments

Project Milestones

Timeline

- Task 1 – Documentation
 - 28/03 – 19/04
- Task 2 – Code Implementation
 - 20/04 – 31/05
- Task 3 – Final Code
 - 01/06 – 07/06
- Task 4 – Containerization (optional)
 - 08/06 – 22/06

Task 1 - Documentation

- Analyze the system based on the provided OpenAPI (Swagger) specification.
- Produce the following documents:
 - Requirements Document
 - Effort Estimation

Task 2 – Implementation and testing

- Implement the system according to the OpenAPI (Swagger) specification
- Extend the provided codebase, which will include:
 - A complete execution flow (routes → controllers → services → repository → model) as an example
 - Example tests at all levels: unit, integration, and end-to-end (E2E)
- Complete the missing features based on the Swagger definition
- Write your own tests to ensure full coverage.
 - Target: coverage as close as possible to 100%

Task 3 – Final Code Verification

- A complete end-to-end test suite will be provided
- These tests will verify that all features defined in the Swagger specification have been correctly implemented
- You will have to:
 - Run the test suite against your code
 - Fix any issues as needed
 - Submit a final version of the code that passes 100% of the provided tests

Task 4 - Containerization

- Containerize the entire system, composed of backend, frontend and database
- Replace SQLite with MySQL, using the official Docker image
 - An init.sql script will be provided to initialize the database
- Containerize the backend
 - create a Dockerfile for the backend application
 - create-root script must be executed before the backend starts, in order to create a default admin user in the database
- Create a docker-compose.yml file to orchestrate all components
- No changes to the backend code will be necessary:
 - the provided configuration of the backend supports DBMS switching via environment variables set in the docker-compose

Project Structure

- The structure of the project evolves progressively through the assigned tasks
- The current architecture and organization are always described in the README.md file
- The README.md also includes:
 - provided development and execution scripts
 - usage instructions
 - useful technical details